

Cómo OpenCode recuerda (o no) lo que haces

Guía para creadores de documentación

JA Palazón Ferrando

2026-06-21

Contenidos

1	¿Qué es OpenCode y por qué debería importarte?	1
2	El problema: cada sesión empieza en blanco	2
3	La memoria básica: las instrucciones permanentes	2
3.1	Instrucciones para tu proyecto	2
3.2	Instrucciones globales (para todos tus proyectos)	3
3.3	Combinando ambos	3
3.4	El límite de este enfoque	3
4	Los <i>skills</i>: conocimiento especializado a la carta	3
5	La memoria avanzada: cuando el asistente recuerda por sí mismo	4
5.1	Sistema de bloques de memoria	4
5.2	Sistema de búsqueda semántica	4
5.3	Sistema de captura automática	5
5.4	Sistema de grafo de conocimiento	5
5.5	Comparativa rápida	5
6	¿Qué estrategia te conviene según tu equipo?	5
7	Conclusión	6
7.1	Referencias	6

1. ¿Qué es OpenCode y por qué debería importarte?

Imagina un asistente que trabaja contigo en la terminal, ayudándote a escribir documentos, estructurar guías, revisar texto y dar formato. Puedes pedirle en lenguaje natural: “pásame esto a markdown”, “revisa la ortografía de este informe”, o “genera una tabla de contenidos para esta guía”.

Eso es OpenCode: un asistente de IA que entiende tu proyecto y trabaja sobre tus archivos.

Pero hay un detalle importante: **cada vez que inicias una conversación, el asistente empieza desde cero**. No recuerda lo que hiciste ayer, ni las preferencias que le dijiste la semana pasada, ni cómo está organizado tu proyecto de documentación.

i ¿Quieres continuar una sesión anterior?

Puedes reiniciar una sesión con el flag `-c` o indicando el número de sesión. Así el asistente recupera el contexto previo sin empezar de cero.

Esta breve guía explica cómo funciona esa memoria, qué opciones tienes para mejorarla y cuál te conviene según tu forma de trabajar.

2. El problema: cada sesión empieza en blanco

Imagina que cada mañana llega un becario nuevo a tu equipo. Es brillante, escribe bien y aprende rápido, pero al día siguiente aparece otro becario que no sabe nada de lo que hicisteis el día anterior. No conoce el manual de estilo, no sabe qué guías estáis escribiendo, ni qué formatos preferís.

Trabajar con OpenCode por defecto es exactamente eso.

Por qué está diseñado así:

- **Privacidad:** OpenCode no guarda nada de tus conversaciones a menos que tú se lo pidas explícitamente.
- **Simplicidad:** sin estado persistente, no hay configuraciones que se rompan ni datos que se corrompan.
- **Flexibilidad:** puedes usar OpenCode para proyectos muy diferentes sin que se mezclen.

Pero para el trabajo real de documentación, necesitas que el asistente recuerde al menos algunas cosas: tu guía de estilo, las convenciones de tu equipo, los formatos que usáis habitualmente.

3. La memoria básica: las instrucciones permanentes

La forma más sencilla de que OpenCode recuerde información es escribirla en un archivo de instrucciones. Es como dejarle una nota al becario antes de que empiece su jornada.

3.1. Instrucciones para tu proyecto

Crea un archivo llamado `AGENTS.md` en el directorio raíz de tu proyecto. Todo lo que escribas ahí, el asistente lo leerá al empezar cada sesión.

`# AGENTS.md (ejemplo para un proyecto de documentación)`

`## Guía de estilo`

- `- Usamos tono formal pero accesible`
- `- Títulos en mayúscula solo la primera letra`
- `- Las listas deben tener línea en blanco entre elementos`

`## Formatos de salida`

- `- Documentos técnicos: Quarto con HTML y PDF`
- `- Guías rápidas: Markdown`
- `- Manuales de usuario: Quarto con EPUB`

`## Convenciones`

- `- Las capturas de pantalla van en /img`
- `- Los diagramas en /assets/diagrams`
- `- Los archivos fuente en /src`

Puedes pedirle a OpenCode que genere este archivo automáticamente con el comando `/init`. Él mismo escaneará tu proyecto y creará un resumen con lo más importante.

3.2. Instrucciones globales (para todos tus proyectos)

Si trabajas en varios proyectos de documentación y quieres que el asistente recuerde siempre ciertas cosas (por ejemplo, tu nombre, tu tono preferido, o reglas generales de estilo), puedes crear un archivo en tu directorio personal:

```
~/.config/OpenCode/AGENTS.md
```

Este archivo se aplica a **todos** los proyectos en los que uses OpenCode. Es útil para:

- Tu forma de trabajar preferida
- Recordatorios generales de estilo
- Preferencias personales de formato

3.3. Combinando ambos

Lo bueno es que ambos archivos se complementan. Las instrucciones globales y las del proyecto se mezclan, así que el asistente recibe toda la información relevante en cada sesión.

Tipo	Dónde se guarda	Para qué sirve
Del proyecto	AGENTS.md en tu proyecto	Reglas específicas de ese proyecto
Globales	~/.config/OpenCode/AGENTS.md	Tus preferencias personales en todos los proyectos

3.4. El límite de este enfoque

Las instrucciones en AGENTS.md son **estáticas**. Si cambias algo, tienes que editarlas a mano. El asistente no aprende de tus conversaciones ni guarda automáticamente los descubrimientos que hace mientras trabaja.

Para eso necesitas niveles más avanzados de memoria.

4. Los *skills*: conocimiento especializado a la carta

Los *skills* (habilidades o competencias) son como fichas de consulta rápida. Contienen instrucciones muy concretas sobre una tarea específica, y el asistente las carga solo cuando las necesita.

Por ejemplo, puedes tener un skill para el formato de tus documentos Quarto. Cuando el asistente vea que estás trabajando con un archivo .qmd, cargará automáticamente las instrucciones de estilo.

```
.OpenCode/skills/estilo-quarto/  
SKILL.md ← "Para documentos Quarto, usa estos formatos..."
```

La ventaja es que el asistente no lleva todo ese conocimiento encima todo el tiempo (lo que ahorra espacio en su “cabeza”), pero lo tiene disponible cuando lo necesita.

Son útiles para:

- Reglas de formato específicas (Quarto, Markdown, LaTeX)
- Guías de estilo para distintos tipos de documento
- Plantillas de estructura recurrente
- Listas de comprobación para revisar documentación

5. La memoria avanzada: cuando el asistente recuerda por sí mismo

Si trabajas con muchos proyectos o equipos grandes, las instrucciones estáticas se quedan cortas. Necesitas que el asistente recuerde automáticamente lo que ha aprendido.

La comunidad de OpenCode ha creado varios sistemas para añadir memoria persistente. Aquí tienes los más relevantes explicados sin tecnicismos.

Más información

Consulta la [documentación de plugins de OpenCode](#) para ver ejemplos prácticos y guías de implementación de cada sistema de memoria.

5.1. Sistema de bloques de memoria

Algunos *plugins* añaden “bloques de memoria” que el asistente puede leer y modificar por sí mismo.

Dónde se guarda la memoria:

Tipo	Se recuerda ...	Ejemplo
Global	En todos los proyectos	“La autora prefiere tono formal”
Del proyecto	Solo en ese proyecto	“Este manual usa convención X para los ejemplos”

Cómo funciona: El asistente tiene tres bloques por defecto:

1. **Comportamiento** — cómo debe actuar y responder el asistente
2. **Preferencias del usuario** — tus costumbres y configuración personal
3. **Contexto del proyecto** — lo específico de cada proyecto

El asistente puede añadir, modificar o borrar información de estos bloques durante la conversación. Así, si descubre algo relevante, lo guarda automáticamente para la próxima sesión.

5.2. Sistema de búsqueda semántica

Otros *plugins* almacenan la memoria en una base de datos que entiende el significado de lo que guardas, no solo las palabras exactas.

Esto significa que si guardas “en este proyecto usamos el formato APA para las referencias”, y más tarde preguntas “¿cómo se citan las fuentes aquí?”, el asistente encontrará esa memoria aunque uses palabras diferentes.

Dos niveles de memoria:

- **Tuya personal:** preferencias que te siguen a cualquier proyecto (“siempre uso tamaño 11pt para los PDFs”)
- **Del proyecto:** cosas específicas de ese proyecto (“este manual usa notas al pie en lugar de citas entre paréntesis”)

5.3. Sistema de captura automática

Hay *plugins* que observan lo que haces y lo guardan ellos solos. Mientras trabajas (lees archivos, haces cambios, ejecutas comandos), el plugin va tomando notas. Luego, con la ayuda de la IA, comprime esas notas en observaciones útiles.

Al empezar una nueva sesión, el asistente recibe un resumen de lo más importante que ocurrió en sesiones anteriores. Es como si el becario llegara cada mañana con un informe de lo que se hizo el día anterior.

5.4. Sistema de grafo de conocimiento

Los más avanzados construyen un “mapa de conceptos” donde conectan ideas: una decisión de formato lleva a un estilo concreto, que a su vez afecta a ciertos tipos de documento. Esto permite al asistente navegar relaciones complejas entre la información que ha aprendido.

5.5. Comparativa rápida

Sistema	Lo bueno	Lo menos bueno	Ideal para
Instrucciones (AGENTS.md)	Sencillo, sin instalación	No aprende solo	Proyectos pequeños, equipos de 1-2 personas
Skills	Ligero, bajo demanda	Hay que crearlos manualmente	Tareas repetitivas con formato fijo
Bloques de memoria	El asistente se actualiza solo	Requiere instalar un plugin	Equipos pequeños que quieren persistencia básica
Búsqueda semántica	Encuentra memorias relacionadas	Más complejo de configurar	Proyectos grandes con mucha documentación
Captura automática	No requiere acción manual	Consume más recursos	Equipos que quieren “configurar y olvidar”
Grafo de conocimiento	Relaciona conceptos complejos	El más complejo	Grandes equipos con documentación interconectada

6. ¿Qué estrategia te conviene según tu equipo?

No necesitas instalar sistemas complicados desde el principio. La mayoría de los equipos de documentación pueden seguir esta progresión:

Nivel 1 — Un proyecto pequeño (1-3 personas)

Crea un **AGENTS.md** con vuestra guía de estilo y las convenciones del proyecto. Es suficiente para la mayoría de los casos. Si usáis Quarto o Markdown, añadid un skill con las reglas de formato.

Nivel 2 — Varios proyectos (equipo de documentación)

Además del **AGENTS.md** de cada proyecto, cread un archivo global con las preferencias del equipo. Así todos los miembros tienen la misma base aunque trabajen en proyectos distintos.

Nivel 3 — Equipo grande o documentación extensa

Instalad un sistema de memoria con búsqueda semántica. Especialmente si manejaís muchos términos técnicos, glosarios, o convenciones que cambian con el tiempo. El asistente encontrará la información relevante aunque uses palabras distintas para preguntar.

Nivel 4 — Documentación interconectada (varios manuales, productos, versiones)

Combinad varios sistemas: instrucciones globales para el equipo, skills para formatos específicos, y un grafo de conocimiento para relacionar conceptos entre distintos manuales y productos.

7. Conclusión

OpenCode no recuerda nada entre sesiones por decisión de diseño, pero ofrece un abanico de opciones para que recuerde exactamente lo que tú necesitas:

- Las **instrucciones** son como un manual de estilo que el asistente lee siempre
- Los *skills* son fichas de consulta rápida para tareas concretas
- Los *plugins* de memoria permiten que el asistente aprenda y recuerde por sí mismo

La clave está en elegir lo que necesitas según tu equipo y tus proyectos. Para un equipo pequeño de documentación, las instrucciones y los skills suelen ser más que suficientes. Para equipos grandes con mucha documentación interconectada, los sistemas de memoria avanzada marcan la diferencia entre repetir explicaciones cada sesión y retomar el trabajo exactamente donde lo dejaste.

7.1. Referencias

- [OpenCode](#) — Asistente de IA que trabaja sobre archivos locales. Guías de configuración y uso.
- [Rules \(AGENTS.md\)](#) — Cómo crear instrucciones personalizadas para OpenCode mediante archivos AGENTS.md.
- [Agent Skills](#) — Sistema de habilidades reutilizables que se cargan bajo demanda.
- [Plugins](#) — Cómo extender OpenCode con plugins para añadir funcionalidades como memoria persistente.
- [OpenCode en GitHub](#) — Repositorio oficial del código fuente.

Curso completo disponible en: palazon.github.io/ocCursoAvanzado